

OSAWARE BASIC — Object-Oriented Programming

A specification for the OOP extension to the OSAWARE BASIC interpreter.

1. Design Principles

This spec adds the four pillars of object-oriented programming to OSAWARE BASIC:

- **Encapsulation** — group data and behavior together; hide internals.
- **Object Usage** — instantiate and interact with objects via members.
- **Inheritance** — derive classes from a parent to share and extend behavior.
- **Polymorphism** — invoke overridden methods on derived classes through base references.

"Less is more"

OSAWARE BASIC is intentionally compact. This OOP extension stays compact too:

- **Single inheritance only.** No multiple inheritance, no interfaces, no mixins.
- **No generics.** No `Of T`.
- **No properties.** Use public fields or method pairs.
- **No events.** No `WithEvents`.
- **No operator overloading.**
- **Instance-only members.** No static / shared class-level state.
- **No auto-generated `IDisposable`.** Define an explicit `Dispose` method if needed.

These can be added later if real programs demand them. The four pillars above deliver everything required for hierarchical, polymorphic, encapsulated design.

Backward compatibility

- Every existing OSAWARE BASIC program keeps running unchanged.

- New keywords (`CLASS` , `END CLASS` , `INHERITS` , `OVERRIDABLE` , `OVERRIDES` , `ME` , `MYBASE` , `NEW` , `NOTHING` , `IS` , `PRIVATE` , `PUBLIC`) only have meaning inside their declared context. Outside, they raise a syntax error.
-

2. Class Declaration — Encapsulation

```
CLASS Tank
    PRIVATE X, Z, HP
    PRIVATE NAME$

    SUB Init (startX, startZ, startName$)
        ME.X = startX : ME.Z = startZ
        ME.HP = 100
        ME.NAME$ = startName$
    END SUB

    PUBLIC FUNCTION IsAlive ()
        IsAlive = (ME.HP > 0)
    END FUNCTION

    PUBLIC SUB TakeHit (damage)
        ME.HP = ME.HP - damage
    END SUB

    PUBLIC FUNCTION GetName$ ()
        GetName$ = ME.NAME$
    END FUNCTION
END CLASS
```

Rules

- A class body opens with `CLASS Name [INHERITS Parent]` and closes with `END CLASS` .
- Field declarations come BEFORE any method declaration.
- Each field is decorated with `PRIVATE` or `PUBLIC` . Default for fields is `PRIVATE` .
- Each method is decorated with `PRIVATE` or `PUBLIC` . Default for methods is `PUBLIC` .

- `ME` is the implicit reference to the current instance, available inside any method body.
- The constructor is a `SUB Init`. It is called automatically when `NEW` is used. If the class does not declare an `Init`, `NEW ClassName()` allocates an instance with default field values.

Field defaults

- Numeric fields default to `0`.
 - String fields default to `""`.
 - Reference fields (`AS ClassName`) default to `NOTHING`.
-

3. Object Usage

```
DIM t AS Tank
t = NEW Tank()
CALL t.Init(320, 480, "Tank-1")

IF t.IsAlive() THEN
    CALL t.TakeHit(25)
END IF

PRINT t.GetName$()
```

Rules

- `DIM x AS ClassName` allocates a typed reference slot, initially `NOTHING`.
 - `NEW ClassName(args)` instantiates and calls `Init(args)` if it exists.
 - Member access uses `obj.member`.
 - Method invocation uses `CALL obj.method(args)`. Function calls read as `obj.method(args)` inside expressions.
 - Assignment between reference variables copies the handle. The original and the copy refer to the same underlying instance.
 - `x = NOTHING` releases the reference.
 - `IF x IS NOTHING THEN ...` tests for the null reference.
 - `IF x IS y THEN ...` tests reference equality.
-

4. Inheritance

```
CLASS HeavyTank INHERITS Tank
    PRIVATE ARMOR

    SUB Init (startX, startZ, startName$, armorPlates)
        CALL MYBASE.Init(startX, startZ, startName$)
        ME.ARMOR = armorPlates
    END SUB

    PUBLIC OVERRIDES SUB TakeHit (damage)
        DIM actual : actual = damage - ME.ARMOR
        IF actual < 0 THEN actual = 0
        CALL MYBASE.TakeHit(actual)
    END SUB
END CLASS
```

Rules

- Single inheritance: `CLASS Child INHERITS Parent`.
- All fields of the parent become available in the child instance.
- All methods of the parent are inherited and dispatched via the vtable.
- The parent constructor is NOT called automatically. The child's `Init` must call `MYBASE.Init(...)` explicitly. This makes initialization order visible and consistent with the BASIC convention of explicit control flow.
- `MYBASE` refers to the parent's bound implementation. `MYBASE.method(args)` skips `OVERRIDES` dispatch and calls the parent's version.

5. Polymorphism

```
CLASS Tank
    ...
    PUBLIC OVERRIDABLE SUB TakeHit (damage)
        ME.HP = ME.HP - damage
```

```

    END SUB
END CLASS

CLASS HeavyTank INHERITS Tank
    PUBLIC OVERRIDES SUB TakeHit (damage)
        ...
    END SUB
END CLASS

DIM t AS Tank
t = NEW HeavyTank()
CALL t.Init(0, 0, "Heavy", 10)
CALL t.TakeHit(50)      ' dispatches to HeavyTank.TakeHit

```

Rules

- A method must be declared `OVERRIDABLE` in the parent before a child may override it.
- A child method must be declared `OVERRIDES` to replace a parent's overridable method. Mismatch (overriding a non-overridable method, or marking `OVERRIDES` without a matching parent) is a compile-time error.
- Dispatch is virtual: the runtime class of the object decides which method runs, not the declared type of the variable.
- A method that is not `OVERRIDABLE` cannot be replaced. Calls to that method always reach the parent's implementation.

6. Reserved Keywords

The following identifiers are reserved by the OOP extension:

`CLASS` · `END CLASS` · `INHERITS` · `OVERRIDABLE` · `OVERRIDES` · `ME` · `MYBASE` · `NEW` · `NOTHING` · `IS` · `PRIVATE` · `PUBLIC`

`PRIVATE`, `PUBLIC`, `ME`, and `MYBASE` are scoped: they may only appear inside `CLASS ... END CLASS` blocks. The interpreter raises a syntax error if they are used at the top level.

`NEW` and `NOTHING` are always reserved.

`IS` is overloaded as the reference-equality operator inside expressions; outside that context the parser falls back to legacy behavior.

7. Implementation Strategy

Compile-time (class registry)

When a program loads, the interpreter scans for `CLASS` declarations and builds a per-class definition:

```
ClassDef {
    name          string
    parent        string | null
    fields        [{ name, type, isPublic }]
    methods       Map<methodName, MethodDef>
    methodVtable  Map<methodName, ResolvedMethod>
}

MethodDef {
    name          string
    kind          "SUB" | "FUNCTION"
    isOverridable boolean
    overrides     boolean
    isPublic      boolean
    paramList     [string]
    startLine     number
    endLine       number
}
```

The vtable for a class is built by:

1. Copying the parent's vtable (when `INHERITS` is present).
2. Adding all of the class's own methods.
3. Replacing entries for methods marked `OVERRIDES`.

The class registry is stored on the kernel as `this._classDefs` and reset on every program RUN.

Runtime (object representation)

Each instantiated object is represented as a host-language record:

```
ObjectInstance {  
    classDef      ref to ClassDef  
    fields        Map<name, value>  
}
```

Instances are stored in `this._objects` keyed by an integer handle. The BASIC-side variable holds the integer handle, so all existing variable-storage code, array indexing, and assignment paths continue to work without modification.

Method dispatch

`CALL obj.method(args)` performs:

1. Read `obj` to get the integer handle.
2. Look up the `ObjectInstance` and its `classDef`.
3. Resolve `method` via `classDef.methodVtable`.
4. Push a SUB frame using the existing `_sub_stack` mechanism, augmented to:
5. Bind `ME` to the current handle.
6. Make the object's fields appear in the method scope under `ME.fieldname`.
7. Execute the method body using the existing tick/interpret loop.
8. On `END SUB / END FUNCTION`, write back any changes to the object's fields and pop the frame.

MYBASE.method dispatch

Each class records a `mybaseVtable` that points at the parent's resolved methods. Calls of the form `MYBASE.method(args)` use `mybaseVtable` instead of `methodVtable`, skipping any OVERRIDES in the current class.

Object lifecycle

- `NEW ClassName(args)` allocates a handle, populates default field values, optionally invokes `Init`, and returns the handle.
- Assignment copies the handle (reference semantics).
- Assignment to `NOTHING` clears the reference.

- On program END, all live objects are disposed.

Coexistence with existing BASIC

- Numeric variables continue to hold numbers; string variables hold strings; object handles are integers that happen to live in a separate handle table.
 - SUB , END SUB , CALL , SHARED , DIM keep their existing semantics outside CLASS blocks.
 - A program that does not use the new keywords pays zero runtime cost and produces no behavioral differences compared to the pre-OOP interpreter.
-

8. Compile-time and Runtime Errors

The interpreter raises clear errors for the following:

- OVERRIDES on a method whose name does not exist in the parent vtable.
- OVERRIDES on a parent method not marked OVERRIDABLE .
- INHERITS of a class that does not exist.
- A cycle in the inheritance chain.
- ME or MYBASE used outside a class method.
- MYBASE in a class with no parent.
- Calling a method on NOTHING .
- DIM x AS ClassName for a class that does not exist.

These are reported to the terminal via the standard appendLine channel with a one-line program-error message that identifies the offending source line.

9. Examples

The OSAWARE distribution ships three example programs that exercise the spec:

- **OOPTANK** — a small Tank class showing encapsulation, Init , public methods, and member access.
- **OOPSHIP** — a Ship class inheriting from Vessel , demonstrating MYBASE , OVERRIDABLE / OVERRIDES , and polymorphic dispatch through a base-class reference.

- **OOPZOO** — a small zoo of `Animal` derivatives with polymorphic `Speak()` calls iterated in a single loop, demonstrating reference-typed variables holding different concrete subclasses.

Each example runs from the OSAWARE shell with `RUN "00PTANK"`, `RUN "00PSHIP"`, `RUN "00PZOO"`.

This document specifies the OOP extension to OSAWARE BASIC. It is intentionally narrow: enough surface area to write hierarchical, polymorphic code in a clear style; not so much that the interpreter loses its compact character.